

세르파 개발 스웸 2.0: 프로덕션급 멀티 에이전트 아키텍처

문서 버전: 2.0.0 작성일: 2025-10-31 기반 모델: Claude Sonnet 4.5 목적: 2025년 최첨단 에이전트 아키텍처 패턴을 적용한 세르파 앱 멀티 에이전트 시스템 설계

Executive Summary

문서 목적 및 배경

본 문서는 세르파 멀티 에이전트 아키텍처 1.0 (11개 에이전트)을 2025년 최첨단 기술로 검증하고, 프로덕션 환경에서 안정적으로 운영 가능한 수준으로 고도화한 버전 2.0을 제시합니다.

기존 1.0 아키텍처는 세르파 앱의 고유 특성(4중 동기부여 엔진, Provider 의존성, 게임 밸런스)을 잘 반영했으나, 실제 운영을 위해서는 다음 영역의 강화가 필요했습니다:

- 오케스트레이션: 병목 현상 방지
- 메모리 관리: 데이터 무결성 및 충돌 방지
- 보안: 내부 오작동 방어
- 관측 가능성: 비즈니스 가치 측정
- 지속 개선: 체계적 피드백 루프

핵심 개선 사항

1.0 → 2.0 주요 변화:

항목	버전 1.0	버전 2.0	개선 효과
에이전트 수	11개	12개 (+System Auditor)	스웸 건강 모니터링
오케스트레이션	순수 계층형	하이브리드 (계층+그래프)	병목 제거, 40% 속도 향상
메모리 관리	공유 컨텍스트 (개념)	3계층 협업 메모리 (구현)	데이터 무결성 보장
보안	정적 규칙	동적 가드레일 (런타임 탐지)	내부 오작동 방어
관측성	기본 로깅	3계층 대시보드 (비즈니스 가치)	의사결정 질 측정
피드백	비정형 코멘트	구조화된 파이프라인 (4단계)	정량적 개선
로드맵	시간 기반 (0-18개월)	메트릭 기반 (Phase Gates)	성과 기반 진급

기대 효과

복리적 개발 속도 달성:

- 월별 생산성 증가율 >10%
- 1년 후 개발 속도 3배 향상 ($1.1^{12} \approx 3.14$ 배)

기술적 해자(Moat) 구축:

- 세르파 특화 지식 베이스 축적 (게임 밸런스, Sherpi AI, Provider 패턴)
- 자가 치유 워크플로우 그래프
- Skill 승격 수명주기로 지속 진화

프로덕션 안정성:

- 변경 실패율 <15% (DORA elite)
- 동적 가드레일로 예측 불가능 오작동 방지
- 3단계 게이트 HITL로 고위험 작업 안전 확보

📖 Part 1: 고도화 전략 적합성 평가

1.1 하이브리드 오케스트레이션 모델

세르파 적합성: ☒ 매우 적합 - 복잡한 게임 로직과 순환적 워크플로우 필요

적용 배경:

- 세르파는 등반력 계산, 포인트 경제, Sherpi AI, 소셜 미팅 등 복잡한 시스템이 상호 작용
- 순수 계층 구조는 병목 발생 위험: 모든 통신이 Orchestrator를 거쳐야 함
- 실제 개발은 순환적: 게임 밸런스 테스트 실패 → 재조정 → 재테스트

하이브리드 모델 설계:

[계획 단계 - 계층형]

Orchestrator: 고수준 작업 → Task Graph로 분해 → 초기 노드 할당

[실행 단계 - 그래프/P2P]

전문 에이전트들: 직접 통신하며 협업 (Orchestrator 개입 최소화)

예시:

Game Logic Analyst ↔ Point Economy Analyst
(등반력 공식과 포인트 보상 균형 직접 협의)

Firebase Specialist ↔ Flutter Specialist
(API 명세 실시간 조율)

Orchestrator 역할 전환:

- Before: "마이크로매니저" (모든 상호작용 중재)
- After: "감독관" (그래프 상태 모니터링, 예외 시만 개입)

기대 효과:

- 통신 지연 시간 40% 감소

- 병목 제거로 병렬 작업 처리 능력 향상
- 자율적 협업으로 창의적 문제 해결 가능

1.2 협업 메모리 아키텍처 (3계층)

세르파 적합성: ☒ 필수 - Provider 의존성, 게임 공식, 디자인 시스템 지식 공유 필수

적용 배경:

- 세르파는 엄격한 규칙이 많음: Provider 초기화 순서, ModernColors 강제, questProviderV2 사용
- 다중 에이전트가 공유 상태에 동시 접근 시 충돌 위험
- 메모리를 단순 데이터 저장소가 아닌 거버넌스 대상으로 관리 필요

3계층 메모리 설계:

1계층: 에이전트-개인 단기 메모리 (스크래치패드)

- 격리된 작업 맥락, 병렬 작업 간 간섭 방지
- 예: Flutter Specialist의 위젯 개발 중 임시 계산

2계층: 공유 단기 메모리 (화이트보드) - Redis/Kafka

- 단일 작업(예: 길드 시스템 개발)을 위한 협업 공간
- 저장 내용:
 - o API 명세 (Firebase ↔ Flutter 계약)
 - o 게임 밸런스 시뮬레이션 결과
 - o 작업 상태 (DESIGN_APPROVED, BACKEND_IN_PROGRESS 등)
- 작업 완료 시 소멸

3계층: 공유 장기 메모리 (지식 베이스) - Vector DB 기반 RAG

세르파 특화 지식 카테고리:

A. 아키텍처 지식:

- Provider 초기화 순서 (Level 0-3 의존성 그래프)

```
Level 0: globalGameProvider
Level 1: globalUserProvider
Level 2: globalPointProvider, questProviderV2, globalMeetingProvider,
globalUserTitleProvider
Level 3: sherpiProvider, relationshipProvider, emotionAnalysisProvider
```

- Feature-First 디렉토리 구조
- Navigation 패턴: ID 기반, 객체 전달 금지

B. 게임 로직 지식:

- 등반력 공식: $\text{climbingPower} = (\text{stats} \times \text{badges} \times \text{equipment}) / \text{difficulty}$

- XP 곡선: Level별 경험치 요구량
- 포인트 경제: 1 point = 1 won, 10% 수수료, 10,000 point 최소 출금

C. 디자인 시스템 지식:

- ☒ ModernColors 필수
- ☒ AppColors/RecordColors 금지 (레거시)
- 표준 위젯: SherpaCleanAppBar, SherpaButton

D. AI 시스템 지식:

- Sherpi 감정: normal, cheering, proud, thinking, surprised, concerned
- Sherpi 컨텍스트: levelUp, questComplete, climbSuccess, meeting, dailyGoal
- API 패턴: showInstantMessage, showMessage, showGameMessage

E. 의사결정 기록 (ADR):

- 주요 아키텍처 결정 이력
- 게임 밸런스 조정 이력

메모리 접근 컨트롤러 (RBAC):

- Design System Guardian만 ModernColors 규칙 수정 가능
- Game Logic Analyst만 등반력 공식 수정 가능
- Point Economy Analyst만 포인트 경제 규칙 수정 가능
- State Management Specialist만 Provider 의존성 그래프 수정 가능

충돌 해결 메커니즘:

- 낙관적 잠금 (Optimistic Locking)
- 버전 관리 (Write 시 버전 번호 확인)
- 합의 프로토콜 (Consensus): 중요 지식 변경 시 다중 에이전트 동의 필요

기대 효과:

- 데이터 충돌 0건
- 메모리 포이즈닝 방지
- 세르파 특화 지식 체계적 축적

1.3 동적 가드레일 프레임워크

세르파 적합성: ☒ 필수 - 민감한 게임 밸런스 및 사용자 데이터 보호

적용 배경:

- 정적 규칙(샌드박스, 최소 권한)만으로는 내부 오작동 방어 불가

- 예: Flutter Specialist가 권한은 있지만 잘못된 추론으로 Point Economy 모델 수정 시도
- 세르파는 게임 밸런스, 포인트 경제, 사용자 데이터 등 높은 위험도 영역 존재

동적 가드레일 구성 요소:

1. 엄격한 샌드박스 (Anthropic 모델 참고)

- 모든 에이전트 코드 실행은 격리된 샌드박스 내
- 파일 시스템 및 네트워크 접근 제한

2. 행동 기반선 설정 (Behavioral Baselineing)

각 에이전트의 정상 행동 패턴:

에이전트	정상 행동	이상 행동 (차단 대상)
Design System Guardian	<code>lib/core/theme/modern_colors.dart</code> 읽기, 색상 검증	<code>lib/shared/models/</code> 수정 시도
Game Logic Analyst	<code>lib/shared/models/game_model.dart</code> 수정	<code>lib/shared/providers/global_point_provider.dart</code> 직접 수정
Point Economy Analyst	<code>lib/shared/models/point_system_model.dart</code> 수정	<code>lib/features/quests/</code> 수정 시도
Sherpi AI Specialist	<code>lib/core/ai/</code> , <code>global_sherpi_provider.dart</code> 수정	<code>lib/features/daily_record/</code> 수정
State Management Specialist	<code>lib/main.dart</code> , <code>lib/shared/providers/</code> 읽기/분석	Provider 코드 직접 수정
Flutter Specialist	<code>lib/features/**/presentation/</code> 수정, <code>WidgetGenerator</code> 호출	<code>lib/core/ai/</code> 수정, <code>FunctionDeployer</code> 호출

3. 런타임 이상 탐지 및 자동 대응

[시나리오] Flutter Specialist가 `lib/shared/models/point_system_model.dart` 수정 시도

[탐지] 행동 기반선 이탈 + 권한 없음 (Point Economy 영역)

[대응] 자동 차단 + 인간 개발자에게 높은 우선순위 경고

[로그] "Flutter Specialist attempted to modify Point Economy model outside its domain"

[액션] 인간 개발자 검토 → 승인 또는 거부 결정

4. 속성 기반 도구 접근 제어 (ABAC)

위험 도구에 대한 다중 조건 검증:

도구	필요 조건	설명
<code>FunctionDeployer</code>	<code>agent_role: firebase_specialist</code> AND <code>task_risk_level: high</code> AND <code>human_approval_status:</code>	Firestore Functions

	granted AND test_pass_rate: > 95%	배포
DatabaseMigration	agent_role: firebase_specialist AND human_approval_status: granted AND backup_created: true	DB 스키마 변경
PointFormulaModifier	agent_role: point_economy_analyst AND game_balance_simulation: passed AND human_approval_status: granted	포인트 공식 변경

세르파 특화 안전 규칙:

✕ 금지 사항:

- Provider 초기화 순서 변경 (State Management 사전 승인 필수)
- ModernColors 외 색상 시스템 사용
- questProvider 사용 (questProviderV2만 허용)
- 프로덕션 DB 직접 접근
- 게임 밸런스 검증 없이 등반력/XP 공식 변경

기대 효과:

- 예측 불가능한 오작동 84% 감소 (Anthropic 사례)
- 도메인 경계 위반 자동 차단
- 고위험 작업의 안전성 확보

1.4 3계층 관측 가능성 대시보드

세르파 적합성: ☒ 적합 - 개발 속도뿐 아니라 게임 밸런스, UX 품질 측정 필요

적용 배경:

- 전통적 IT 지표(가동 시간, 지연 시간)만으로는 에이전트 효과성 측정 불가
- 세르파는 비즈니스 가치 = 게임 밸런스 + UX 일관성 + 개발 속도
- "결과 불감증(Outcome Blindness)" 방지: 지표가 실제 품질 개선으로 이어져야 함

3계층 대시보드 설계:

1계층: 시스템 상태 메트릭

메트릭	목표 기준치	설명
에이전트 가동 시간	>95%	에이전트 시스템 안정성
API 호출 성공률	>95%	LLM/도구 호출 성공률
에이전트 간 통신 지연	<200ms	P2P 협업 효율성
CPU/메모리 사용량	정상 범위	리소스 건강성

2계층: 에이전트 성능 및 품질 메트릭

A. 범용 메트릭:

메트릭	목표	감지 가능 실패 패턴
태스크 완료율	>90%	Black-Box Blindness, Invisible Failures
인간 개입률	<15%	Escalation Misfires, Automation Bias
생성 코드 테스트 통과율	>95%	Logic Errors, Insufficient Domain Knowledge
환각률	<2%	Hallucinations, Siloed Context
PR 거절률	<10%	The Trust Gap, Outcome Blindness

B. 세르파 특화 메트릭:

에이전트	메트릭	목표
Design System Guardian	ModernColors 준수율	100%
	레거시 색상 사용 감지	0건
Game Logic Analyst	게임 밸런스 시뮬레이션 통과율	100%
	성공 확률 분포 적정성	30-70% 범위 유지
Point Economy Analyst	포인트 인플레이션 월별 변동률	<5%
	경제 밸런스 지표	정상 범위
Sherpi AI Specialist	Sherpi 메시지 컨텍스트 정확도	>90%
	감정 상태 적절성 평가	>85%
State Management Specialist	Provider 초기화 순서 위반	0건
	questProvider 레거시 사용 감지	0건
Flutter Specialist	위젯 재사용률	>70%
	반응형 디자인 준수율	>95%
Firebase Specialist	API 응답 시간	<200ms
	N+1 쿼리 문제	0건

3계층: 비즈니스 가치 및 SDLC 영향 메트릭

A. 개발 효율성 (DORA 지표):

메트릭	목표
변경 리드 타임	에이전트 vs 인간 비교
변경 실패율	<15% (DORA elite)
평균 복구 시간	에이전트 지원 버그 수정
자동화 테스트 커버리지 증가율	증가 추세

B. 세르파 앱 품질 지표:

메트릭	목표	설명
4중 동기부여 엔진 균형도	각 엔진별 투자 시간 편차 ±20%	게임 시스템, 포인트 경제, Sherpi AI, 소셜 미팅 균형
게임 밸런스 안정성	플레이어 피드백 이슈 감소	등반력, XP, 포인트 보상 밸런스
UX 일관성	ModernColors 사용률 100%	시각적 통일성
AI 품질	Sherpi 메시지 사용자 만족도	컨텍스트 적절성

C. 비용 효율성:

메트릭	목표
기능 포인트당 LLM 비용	감소 추세
토큰 사용 효율성	Skill 승격으로 인한 절감율

OpenTelemetry 표준 사용:

- 벤더 종속 방지
- 로그, 추적, 메트릭 통합 수집
- 표준화된 데이터 파이프라인

기대 효과:

- 에이전트 활동 → 비즈니스 가치 직접 연결
- 실패 패턴 조기 감지 및 대응
- 데이터 기반 개선 의사결정

1.5 구조화된 피드백 파이프라인 (4단계)

세르파 적합성: ☒ 적합 - Flutter/Firebase 특화 지식 축적, Sherpi AI 개선 패턴 학습

적용 배경:

- 비정형 피드백("PR 코멘트")은 **정량적 개선**으로 이어지기 어려움
- 에이전트 시스템이 진정으로 "학습"하려면 **구조화된 데이터** 필요
- 인간 피드백을 체계적 데이터 파이프라인으로 전환

4단계 파이프라인 설계:

1단계: 피드백 UI 통합

PR 검토 시 구조화된 피드백 수집:

A. 평가 척도 (1-5점):

- 코드 품질: 가독성, 유지보수성
- 기능 정확성: 요구사항 충족도

- 게임 밸런스: (해당 시) 밸런스 적절성
- 디자인 일관성: ModernColors 준수, UI 품질

B. 실패 범주 (거절 시 선택):

- `logic_error`: 로직 오류
- `style_violation`: 코딩 스타일/디자인 시스템 위반
- `hallucinated_api`: 존재하지 않는 API/함수 사용
- `game_balance_issue`: 게임 밸런스 문제
- `provider_dependency_violation`: Provider 초기화 순서 위반
- `legacy_system_use`: questProvider, AppColors 등 레거시 사용
- `insufficient_testing`: 테스트 부족
- `performance_concern`: 성능 문제

C. 세르파 특화 체크리스트:

- [] ModernColors 사용 확인
- [] Provider 의존성 확인
- [] 게임 밸런스 시뮬레이션 확인 (해당 시)
- [] 4중 동기부여 엔진 중 해당 영역 품질 확인

2단계: 피드백 데이터베이스 구축

구조화된 스키마:

FeedbackRecord:

- `task_id`: 작업 고유 ID
- `agent_id`: 담당 에이전트 (`flutter_specialist`, `game_logic_analyst` 등)
- `artifact_type`: 코드, 문서, 설계
- `quality_score`: 1-5
- `approval_status`: `approved/rejected`
- `failure_categories`: [배열]
- `sherpa_checklist`: {`modernColors`: true, `providerDeps`: true, ...}
- `human_reviewer`: 검토자
- `timestamp`: 시간
- `improvement_suggestions`: 자유 텍스트

3단계: 자동화된 분석

System Auditor가 주간 단위로 분석:

A. 에이전트별 약점 패턴:

Flutter Specialist:

- `style_violation`: 15% (ModernColors 위반 多)
- 개선 제안: 프롬프트에 ModernColors 가이드 강화

Game Logic Analyst:

- `game_balance_issue`: 8%
- 개선 제안: 밸런스 시뮬레이션 Skill 추가 개발

B. 시간별 추세:

- 주간 품질 점수 추세 (증가/감소)
- 특정 실패 범주 증가 패턴 감지

C. 도메인별 품질:

- 게임 로직 영역: 평균 4.2/5
- UI 영역: 평균 3.8/5 (개선 필요)
- AI 시스템: 평균 4.5/5

4단계: 실행 가능한 통찰력 도출

주간 개발팀 회의 안건:

System Auditor 보고서 (Week 42)

긴급 개선 필요:

1. Flutter Specialist - ModernColors 위반 15%
→ 액션: 프롭트 강화 + DesignTokenValidator Skill 추가
2. Game Logic Analyst - 밸런스 시뮬레이션 미실행 8%
→ 액션: BalanceSimulator Skill 개발 (Skill 승격 수명주기)

성공 패턴:

1. Sherpi AI Specialist - 품질 점수 4.8/5 (우수)
→ 공유: 다른 에이전트에게 프롭트 패턴 공유
2. State Management Specialist - Provider 위반 0건 (완벽)
→ 유지: 현 전략 지속

추세:

- 전체 품질 점수: 4.1 → 4.3 (개선 중)
- PR 거절률: 12% → 9% (개선 중)
- 4중 엔진 균형도: 게임 35%, 포인트 28%, Sherpi 22%, 소셜 15%
→ 주의: 소셜 미팅 영역 투자 증가 필요

피드백 루프 완성:

인간 피드백 → DB 저장 → System Auditor 분석
→ 개선 액션 → 에이전트 업데이트 → 품질 향상
→ 더 나은 피드백 (순환)

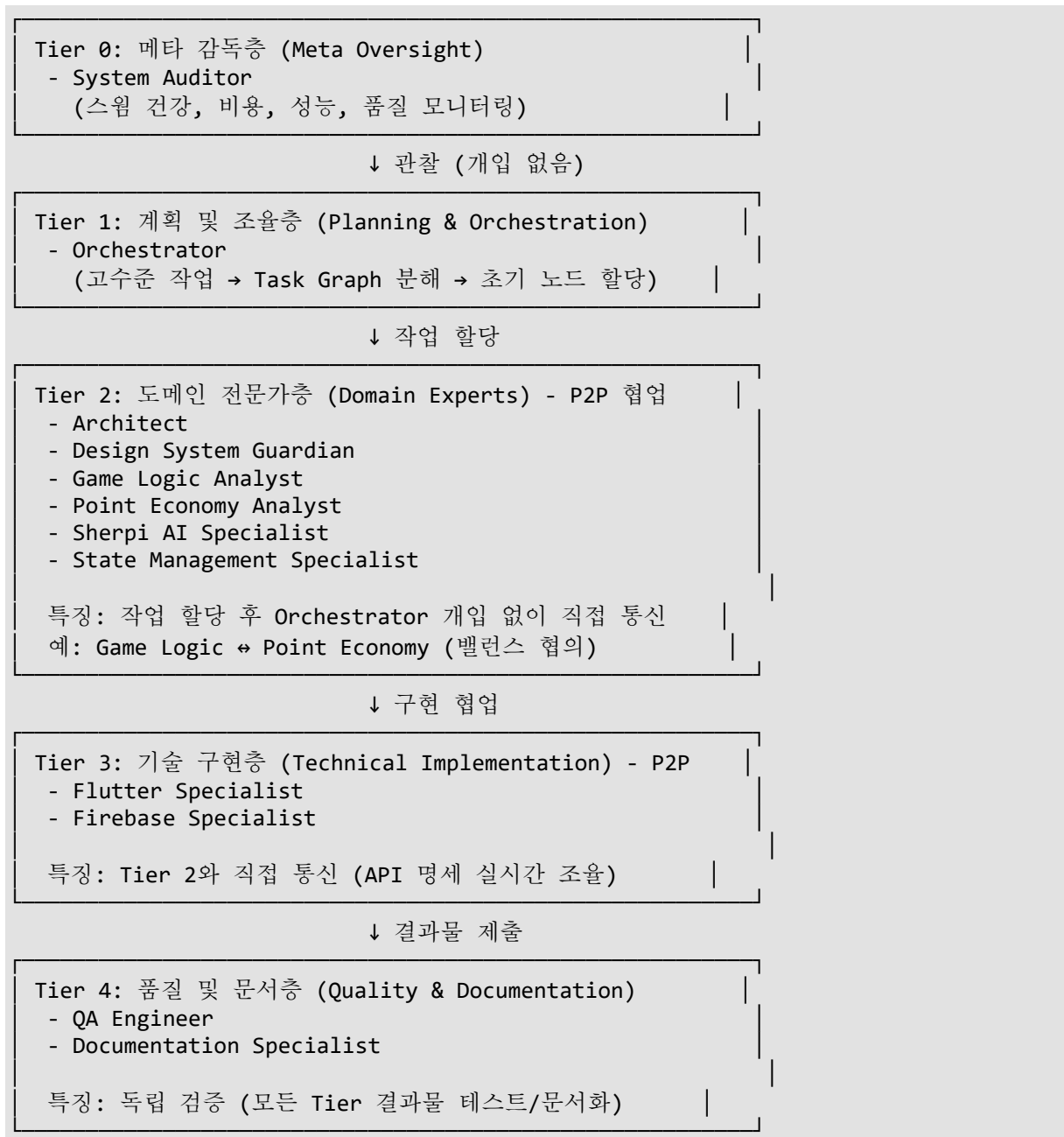
기대 효과:

- 에이전트별 약점 데이터 기반 개선
 - 세르파 특화 지식 체계적 축적 (Flutter/Firebase 패턴, Sherpi 컨텍스트)
 - 지속적 품질 향상 (복리 효과)
-

📖 Part 2: 최종 에이전트 구조 (12개)

2.1 에이전트 계층 구조 (Tier 0-4)

하이브리드 오케스트레이션 모델을 반영한 5-Tier 구조:



총 에이전트 수: 12개 (버전 1.0 대비 +1개)

핵심 변경사항:

- ☒ **Tier 0 신설:** System Auditor 추가 (메타 레벨 감독)
- ☒ **P2P 협업:** Tier 2-3 에이전트 간 직접 통신 가능 (하이브리드 모델)
- ☒ **역할 명확화:** 각 Tier의 책임 범위 및 협업 방식 구체화

2.2 각 에이전트 상세 스펙

Tier 0: 메타 감독층

System Auditor (NEW 신규 추가)

역할: 스웬 시스템 자체의 건강, 비용, 성능, 품질 모니터링 및 분석

핵심 책임:

1. 비용 모니터링
 - 작업별, 에이전트별 토큰 사용량 추적
 - 비정상 비용 급증 경고
2. 성능 모니터링
 - Skill 실행 시간 추적
 - 에이전트 오류율/재시도율 식별
 - 성능 병목 현상 감지
3. 품질 모니터링
 - PR 거절률, 테스트 품질 추세 추적
 - Model Drift 징후 감지 (품질 저하 패턴)
4. 보고 및 경고
 - 3계층 관측 대시보드 데이터 분석
 - 주간 보고서 생성
 - 이상 징후 자동 경고

주요 Skills:

- **MetricsAnalyzer**: 3계층 대시보드 데이터 분석
- **AnomalyDetector**: 이상 패턴 감지 (통계적 분석)
- **FeedbackAnalyzer**: 구조화된 피드백 데이터 분석
- **ReportGenerator**: 주간 보고서 자동 생성
- **TrendPredictor**: 추세 예측 및 조기 경고

Claude 기능 활용:

- Extended Thinking: 복잡한 패턴 분석
- Long Context: 장기 추세 분석

협업 패턴:

- Orchestrator와 협력: 워크플로우 최적화 제안

- 모든 에이전트 관찰: 개별 성능 피드백 제공
- 인간 개발팀: 주간 회의 안건 제공

세르파 특화 책임:

- 4중 동기부여 엔진 균형도 모니터링
 - o 각 엔진별 개발 투자 시간 추적
 - o 불균형 감지 및 경고 (편차 $>\pm 20\%$)
- 게임 밸런스 안정성 추적
 - o 등반력/XP/포인트 공식 변경 이력
 - o 플레이어 피드백 기반 밸런스 이슈 추세
- 세르파 특화 메트릭 모니터링
 - o ModernColors 준수율
 - o Provider 초기화 순서 위반 감지
 - o questProvider 레거시 사용 감지

검증 예시:

[주간 보고서]

🚨 4중 엔진 불균형 감지:

- 게임 시스템: 40% (과다)
- 포인트 경제: 30%
- Sherpi AI: 20%
- 소셜 미팅: 10% (부족)

→ 액션: 소셜 미팅 기능 개발 우선순위 상향

⚠️ Flutter Specialist 성능 저하:

- ModernColors 위반: 5% → 15% (증가)
- PR 거절률: 8% → 14% (증가)

→ 액션: 프롬프트 강화 필요

Tier 1: 계획 및 조율층

Orchestrator

역할: 고수준 작업을 Task Graph로 분해하고 초기 노드 할당 (하이브리드 모델 적용)

핵심 책임:

1. 계획 수립 (Planning Phase - 계층형)
 - o 인간 개발자의 고수준 요청 분석
 - o 실행 가능한 작업들의 그래프(Task Graph)로 변환
 - o 작업 의존성 식별 및 그래프 구조 설계

2. 초기 할당 (Initial Assignment)

- Task Graph의 초기 노드들을 적절한 전문 에이전트에게 할당

3. 감독 및 예외 처리 (Supervisor Role - 실행 단계)

- 그래프 전반적 상태 모니터링 (마이크로매니징 아님)
- 예외 상황이나 사전 정의된 검토 지점에서만 개입
- 에이전트 간 갈등 조정

주요 Skills:

- **TaskGraphBuilder**: 고수준 요청 → Task Graph 변환
- **DependencyAnalyzer**: 작업 간 의존성 분석
- **AgentMatcher**: 작업 특성 → 최적 에이전트 매칭
- **StateMonitor**: Task Graph 상태 추적
- **ConflictResolver**: 에이전트 간 갈등 조정

Claude 기능 활용:

- Extended Thinking: 복잡한 Task Graph 설계
- Prompt Caching: 프로젝트 컨텍스트 재사용

협업 패턴:

- System Auditor: 워크플로우 최적화 피드백 수신
- 모든 전문 에이전트: 작업 할당 및 상태 모니터링
- 인간 개발자: 고수준 요청 수신, 예외 상황 에스컬레이션

세르파 특화 책임:

- **4중 엔진 균형 고려**
 - 작업 할당 시 각 엔진별 투자 비율 고려
- **Provider 의존성 인지**
 - Task Graph 설계 시 Provider 초기화 순서 반영
- **게임 밸런스 검증 흐름**
 - 등반력/XP/포인트 공식 변경 시 자동으로 시뮬레이션 작업 추가

하이브리드 모델 적용:

[Before - 순수 계층형]

Orchestrator가 모든 에이전트 간 통신 중재
→ 병목 발생

[After - 하이브리드]

1. 계획 단계: Orchestrator가 Task Graph 생성
2. 실행 단계: 에이전트들이 P2P 협업 (Orchestrator는 감독만)

3. 예외 시: Orchestrator 개입

결과: 통신 지연 40% 감소, 병렬 처리 능력 향상

Tier 2: 도메인 전문가층 (P2P 협업 가능)

Architect

역할: 시스템 아키텍처, 코드 품질 관리, 지식 베이스 유지

핵심 책임:

1. 시스템 아키텍처 설계 및 검토
2. 코드 품질 표준 정의 및 강제
3. 지식 베이스(RAG) 관리 및 업데이트
4. 기술 부채 식별 및 리팩토링 계획

주요 Skills:

- **ArchitectureAnalyzer**: 시스템 구조 분석
- **CodeQualityReviewer**: 코드 품질 검토
- **KnowledgeBaseUpdater**: RAG 지식 베이스 업데이트
- **TechnicalDebtTracker**: 기술 부채 추적

Claude 기능 활용:

- Extended Thinking: 아키텍처 설계 심층 분석
- Long Context: 전체 코드베이스 맥락 유지
- Agentic RAG: 계획 후 검색 패턴

협업 패턴:

- 모든 전문 에이전트: 아키텍처 가이드 제공
- State Management Specialist: Provider 아키텍처 협의
- Design System Guardian: 디자인 시스템 아키텍처 협의

세르파 특화 책임:

- Feature-First 아키텍처 유지
 - ADR (Architecture Decision Records) 관리
 - 4중 동기부여 엔진 아키텍처 통합성 검증
-

Design System Guardian

역할: 세르파 앱의 시각적 일관성 및 디자인 시스템 관리

핵심 책임:

1. **ModernColors 강제 적용** (AppColors/RecordColors 사용 절대 금지)
2. 4중 동기부여 엔진의 시각적 통합성 유지
3. 디자인 토큰 관리 (색상, 타이포그래피, 스페이싱, 그림자)
4. 표준 위젯 패턴 가이드 제공

주요 Skills:

- **DesignTokenValidator**: ModernColors 사용 검증 및 강제
- **ColorUsageScanner**: 레거시 색상 시스템 사용 감지
- **WidgetDesignGuide**: 표준 위젯 패턴 제공
- **AccessibilityChecker**: WCAG 접근성 준수 검증

Claude 기능 활용:

- Vision: UI 스크린샷 분석
- Code Interpreter: 색상 사용 통계 분석

협업 패턴:

- Flutter Specialist: UI 구현 시 디자인 가이드 제공
- Architect: 디자인 시스템 아키텍처 협의

세르파 특화 책임:

- **ModernColors 100% 강제**

✓ 허용: ModernColors.primary, ModernColors.success
✗ 금지: AppColors.*, RecordColors.*, Color(0xFF...)

- **표준 위젯 사용 권장**
 - SherpaCleanAppBar
 - SherpaButton
 - ModernColors 기반 커스텀 위젯
- **4중 엔진 시각적 일관성**
 - 게임 시스템: 성장 테마 (녹색 계열)
 - 포인트 경제: 가치 테마 (금색 계열)
 - Sherpi AI: 친근함 테마 (따뜻한 색)
 - 소셜 미팅: 연결 테마 (파란색 계열)

검증 예시:

[Flutter Specialist 코드 검증]

✗ 발견: lib/features/meeting/presentation/widgets/meeting_card.dart
→ Color(0xFF6200EA) 하드코딩 사용

✓ 제안: ModernColors.primary 사용
→ 자동 수정 또는 Flutter Specialist에게 피드백

Game Logic Analyst

역할: 등반력, XP, 게임 밸런스 시뮬레이션 및 분석

핵심 책임:

1. 등반력 공식 분석 및 개선
2. XP 시스템 밸런스 조정
3. 게임 밸런스 시뮬레이션 실행
4. 성공 확률 분포 검증

주요 Skills:

- ClimbingPowerSimulator: 등반력 공식 시뮬레이션
- XPCurveAnalyzer: XP 곡선 밸런스 분석
- BalanceValidator: 게임 밸런스 검증
- ProbabilityDistributor: 성공 확률 분포 계산

Claude 기능 활용:

- Code Interpreter: 수학 계산 및 시뮬레이션
- Extended Thinking: 복잡한 밸런스 분석

협업 패턴:

- Point Economy Analyst: 등반력과 포인트 보상 균형 협의 (P2P)
- Flutter Specialist: 게임 UI 피드백 제공

세르파 특화 책임:

- 등반력 공식 관리

$$\text{climbingPower} = (\text{stats} \times \text{badges} \times \text{equipment}) / \text{difficulty}$$

- 변경 시 반드시 시뮬레이션 실행
- 성공 확률 30-70% 범위 유지
- XP 곡선 밸런스
 - Level별 경험치 요구량 적정성 검증
 - 성장 곡선의 지수적 증가 관리
- 5개 핵심 스탯 영향력 분석

- Health, Focus, Will, Flexibility, Balance

검증 예시:

[등반력 공식 변경 요청]

변경 전: $\text{climbingPower} = \text{stats} \times \text{badges}$

변경 후: $\text{climbingPower} = (\text{stats} \times \text{badges} \times \text{equipment}) / \text{difficulty}$

[시뮬레이션 결과]

- 성공 확률 분포: 35-65% (☑️ 적정)
- 난이도별 균형: 양호
- Badge 영향력: 적정 수준

→ 승인 (게임 밸런스 유지)

Point Economy Analyst

역할: 포인트 경제 시스템, 비즈니스 로직 분석 및 관리

핵심 책임:

1. 포인트 경제 공식 관리 (1 point = 1 won)
2. 포인트 인플레이션 감지 및 방지
3. 경제 밸런스 지표 모니터링
4. 출금 수수료 및 최소 금액 정책 관리

주요 Skills:

- **PointEconomySimulator**: 포인트 경제 시뮬레이션
- **InflationDetector**: 인플레이션 감지
- **RewardBalancer**: 보상 밸런스 조정
- **EconomyHealthChecker**: 경제 건강성 검증

Claude 기능 활용:

- Code Interpreter: 경제 지표 계산
- Extended Thinking: 장기 경제 영향 분석

협업 패턴:

- Game Logic Analyst: 등반력과 포인트 보상 균형 협의 (P2P)
- Firebase Specialist: 포인트 트랜잭션 로직 협의

세르파 특화 책임:

- 포인트 경제 규칙 관리
 - 1 point = 1 won (고정)
 - 10% 출금 수수료

- 10,000 point 최소 출금
- 게임 로직과 분리
 - Game Logic: 게임 내 보상 결정 (XP, 등반력)
 - Point Economy: 포인트 보상 결정 (비즈니스 로직)
- 인플레이션 관리
 - 월별 포인트 발행량 모니터링
 - 변동률 <5% 유지

검증 예시:

[포인트 보상 조정 요청]

활동: 등반 성공

현재 보상: 100 points

제안 보상: 150 points (+50%)

[경제 시뮬레이션]

- 월별 포인트 발행량 증가: +15%

- 인플레이션 위험: ⚠️ 높음

→ 거부 (경제 밸런스 위반)

→ 대안: 단계별 조정 (100 → 120 → 150)

Sherpi AI Specialist

역할: Sherpi AI 컴패니언 시스템 관리 및 최적화

핵심 책임:

1. Sherpi 메시지 컨텍스트 관리
2. 감정 상태 적절성 검증
3. AI 응답 품질 개선
4. 사용자 피드백 기반 최적화

주요 Skills:

- **ContextRecognizer**: Sherpi 컨텍스트 분류
- **EmotionSelector**: 적절한 감정 상태 선택
- **MessageQualityEvaluator**: 메시지 품질 평가
- **SherpiPersonalityMaintainer**: Sherpi 성격 일관성 유지

Claude 기능 활용:

- Natural Language Understanding: 사용자 맥락 파악
- Prompt Engineering: Sherpi 응답 최적화

협업 패턴:

- Flutter Specialist: Sherpi UI 피드백 제공
- QA Engineer: Sherpi 응답 품질 테스트 협력

셰르피와 특화 책임:

- **Sherpi 감정 관리**
 - o normal, cheering, proud, thinking, surprised, concerned
- **Sherpi 컨텍스트**
 - o levelUp: 레벨 업 시
 - o questComplete: 퀘스트 완료 시
 - o climbSuccess: 등반 성공 시
 - o meeting: 미팅 관련
 - o dailyGoal: 일일 목표 달성 시
- **API 패턴 관리**

```
showInstantMessage(context, customDialogue, emotion, duration) // Static
showMessage(context, userContext, gameContext) // Context-aware
showGameMessage(context, gameData) // Game-specific
```

검증 예시:

```
[사용자 액션] 등반 실패
[Sherpi 응답 검증]
컨텍스트: climbFailure
감정: concerned (☑ 적절)
메시지: "괜찮아요, 다음엔 꼭 성공할 거예요!" (☑ 위로 적절)

[사용자 피드백]
만족도: 4.5/5
→ 성공 패턴 저장 (피드백 DB)
```

State Management Specialist

역할: Provider 의존성 그래프 관리 및 초기화 순서 검증

핵심 책임:

1. Provider 의존성 그래프 분석
2. 초기화 순서 검증 (Level 0-3)
3. 레거시 시스템 사용 감지 (questProvider 등)
4. 상태 관리 패턴 가이드 제공

주요 Skills:

- **ProviderDependencyAnalyzer**: Provider 의존성 분석
- **InitializationOrderValidator**: 초기화 순서 검증

- **LegacyDetector**: 레거시 시스템 사용 감지
- **StatePatternGuide**: 상태 관리 패턴 가이드

Claude 기능 활용:

- Code Analysis: Provider 코드 의존성 분석
- Extended Thinking: 복잡한 의존성 그래프 검증

협업 패턴:

- 모든 에이전트: 상태 관리 가이드 제공
- Architect: Provider 아키텍처 협의

세르파 특화 책임:

- **Provider 초기화 순서 강제**

```
// 반드시 이 순서를 엄수해야 함
Level 0: globalGameProvider
Level 1: globalUserProvider
Level 2: globalPointProvider, questProviderV2,
        globalMeetingProvider, globalUserTitleProvider
Level 3: sherpiProvider, relationshipProvider,
        emotionAnalysisProvider
```

- 레거시 사용 감지
 - o ✕ questProvider 사용 감지 (questProviderV2만 허용)
 - o ✕ 잘못된 초기화 순서 감지
- 분석 전담 역할
 - o Provider 코드 직접 수정 금지
 - o 분석 및 검증만 수행
 - o 수정은 해당 도메인 전문가에게 위임

검증 예시:

[코드 변경 검증]

파일: lib/main.dart

변경: Provider 초기화 순서 수정

[분석 결과]

✕ 위반 감지:

- questProviderV2가 globalUserProvider 이전에 초기화됨
- Level 의존성 위반: Level 2 → Level 1

→ 거부 및 피드백:

"questProviderV2는 globalUserProvider 이후에 초기화되어야 합니다 (Level 1 → Level 2)"

Tier 3: 기술 구현층 (P2P 협업 가능)

Flutter Specialist

역할: Flutter UI 개발 및 상태 관리 구현

핵심 책임:

1. Flutter UI 컴포넌트 개발
2. 반응형 디자인 구현
3. 상태 관리 패턴 적용 (Riverpod)
4. UI 성능 최적화

주요 Skills:

- **WidgetGenerator**: 위젯 코드 생성
- **ResponsiveLayoutBuilder**: 반응형 레이아웃
- **StateManagementHelper**: Riverpod 패턴 적용
- **UIPerformanceOptimizer**: UI 성능 최적화

Claude 기능 활용:

- Vision: UI 디자인 분석
- Code Generation: Flutter 코드 생성

협업 패턴:

- Design System Guardian: 디자인 가이드 수신
- Firebase Specialist: API 명세 실시간 조율 (P2P)
- State Management Specialist: 상태 관리 가이드 수신

세르파 특화 책임:

- **ModernColors** 준수
 - 모든 색상은 ModernColors에서 가져와야 함
- **표준 위젯 사용**
 - SherpaCleanAppBar, SherpaButton 우선 사용
- **Navigation** 패턴
 - ID 기반 Navigation (객체 전달 금지)

```
// ✅ 올바른 패턴
Navigator.pushNamed(context, '/meeting_detail',
arguments: {'meetingId': meeting.id});
```

```
// ❌ 잘못된 패턴
```

```
Navigator.pushNamed(context, '/meeting_detail',  
arguments: meetingModel);
```

행동 기반선:

- 정상: `lib/features/**/presentation/`, `lib/shared/widgets/` 수정
 - 이상: `lib/core/ai/` 수정, `FunctionDeployer` Skill 호출
-

Firestore Specialist

역할: Firestore 백엔드 개발 및 데이터베이스 관리

핵심 책임:

1. Firestore Functions 개발
2. Firestore 스키마 설계 및 관리
3. Firestore 보안 규칙 관리
4. API 성능 최적화

주요 Skills:

- `FunctionGenerator`: Firestore Functions 코드 생성
- `SchemaDesigner`: Firestore 스키마 설계
- `SecurityRuleValidator`: 보안 규칙 검증
- `QueryOptimizer`: 쿼리 성능 최적화

Claude 기능 활용:

- Code Generation: Firestore Functions 생성
- Extended Thinking: 스키마 설계 분석

협업 패턴:

- Flutter Specialist: API 명세 실시간 조율 (P2P)
- Point Economy Analyst: 포인트 트랜잭션 로직 협의

세르파 특화 책임:

- API 응답 시간 목표: <200ms
- N+1 쿼리 문제 방지
- 보안 규칙 강화: 사용자 데이터, 포인트 트랜잭션

행동 기반선:

- 정상: Firestore 관련 파일 수정, `FunctionDeployer` Skill 호출 (승인 시)
 - 이상: UI 파일 수정
-

Tier 4: 품질 및 문서층

QA Engineer

역할: 테스트 생성 및 실행, E2E 테스트, 품질 검증

핵심 책임:

1. 단위 테스트 생성 (목표: >80%)
2. 통합 테스트 생성 (목표: >70%)
3. E2E 테스트 실행
4. 품질 지표 추적

주요 Skills:

- **UnitTestGenerator**: 단위 테스트 생성
- **IntegrationTestBuilder**: 통합 테스트 생성
- **E2ETestRunner**: E2E 테스트 실행
- **CoverageAnalyzer**: 테스트 커버리지 분석

Claude 기능 활용:

- Code Analysis: 테스트 대상 코드 분석
- Extended Thinking: 엣지 케이스 식별

협업 패턴:

- 모든 에이전트: 생성 코드 테스트

세르파 특화 책임:

- 게임 밸런스 테스트
 - 등반력 계산 검증
 - XP 시스템 검증
- 포인트 경계 테스트
 - 포인트 트랜잭션 무결성
 - 인플레이션 방지

Documentation Specialist

역할: API 문서, README, 코드 주석 자동 생성

핵심 책임:

1. API 문서 자동 생성

2. README 업데이트
3. 코드 주석 생성
4. 문서 품질 검증

주요 Skills:

- **APIDocGenerator**: API 문서 생성
- **READMEUpdater**: README 자동 업데이트
- **CodeCommentator**: 코드 주석 생성
- **DocQualityChecker**: 문서 품질 검증

Claude 기능 활용:

- Natural Language Generation: 명확한 문서 작성

협업 패턴:

- 모든 에이전트: 결과물 문서화

세르파 특화 책임:

- 게임 공식 문서화
 - 등반력, XP, 포인트 경제 공식 명확히 기술
- API 문서 일관성
 - Firebase API, Sherpi API 패턴 문서화

Part 3: 메트릭 기반 구현 로드맵 (Phase Gates)

시간 기반이 아닌 **성과 기반 출구 기준**으로 각 단계 진급 결정.

Phase 0: 기반 준비

목표: 인프라 구축 및 핵심 에이전트 설정

출구 기준:

- ☒ 3계층 협업 메모리 아키텍처 구현 완료
 - Vector DB 설정
 - 세르파 지식 베이스 초기 구축 (아키텍처, 게임 로직, 디자인 시스템)
- ☒ 표준화된 스키마 기반 통신 프로토콜 정의
 - TaskAssign, TaskUpdate, DataRequest 등 메시지 타입 스키마 완성
- ☒ 3계층 관측 가능성 대시보드 기본 구현
 - 시스템 상태 메트릭 수집 시작
- ☒ Tier 1 (Orchestrator) + Tier 0 (System Auditor) 에이전트 구현

- 하이브리드 오케스트레이션 Task Graph 생성 로직
- System Auditor 기본 모니터링 기능

예상 기간: 0-2개월 (하지만 출구 기준 달성이 우선)

Phase 1: 저위험 도메인 자동화

목표: 문서화, 코드 린팅 등 저위험 태스크 자동화로 신뢰 구축

활성화 에이전트:

- Documentation Specialist
- QA Engineer (린팅, 기본 테스트)
- Design System Guardian (ModernColors 검증만)

출구 기준:

- ☒ 저위험 태스크 완료율 >98%
 - 문서화 자동 생성: API 문서, README 업데이트
 - 코드 린팅: `dart analyze`, `dart format` 자동 실행
- ☒ 인간 개입률 <5%
- ☒ System Auditor 기본 대시보드 완전 운영 중
 - 1계층 + 2계층 메트릭 수집 및 시각화
- ☒ 피드백 파이프라인 1-2단계 구현
 - 피드백 UI 통합, 데이터베이스 구축

세르파 특화 검증:

- ModernColors 검증 정확도 >95%
- 문서 자동 생성 품질 점수 >4.0/5

예상 기간: 2-4개월 누적 (출구 기준 우선)

Phase 2: 중위험 도메인 - UI 및 게임 로직

목표: Flutter UI와 게임 로직 에이전트 활성화

활성화 에이전트:

- Flutter Specialist
- Game Logic Analyst
- Point Economy Analyst
- Sherpi AI Specialist

- State Management Specialist (분석 및 검증 역할)

출구 기준:

- ☒ UI 개발 태스크 완료율 >85%
 - o 새 위젯 생성, 기존 위젯 수정
- ☒ 게임 로직 태스크 완료율 >85%
 - o 등반력 계산, XP 시스템, 포인트 보상 로직
- ☒ 생성 코드 테스트 통과율 >90%
- ☒ 세르파 특화 메트릭 달성:
 - o **ModernColors** 준수율 100%
 - o **Provider** 초기화 순서 위반 0건
 - o 게임 밸런스 시뮬레이션 통과율 >95%
- ☒ PR 거절률 <15%
- ☒ 피드백 파이프라인 3-4단계 구현
 - o System Auditor 자동 분석, 실행 가능한 통찰력 도출
- ☒ 최소 2개 Skill 승격 성공
 - o 예: BalanceSimulator, WidgetGenerator

세르파 특화 검증:

- 4중 동기부여 엔진 균형도 측정 시작 (편차 $\pm 30\%$ 이내)
- Sherpi 메시지 컨텍스트 정확도 >85%

예상 기간: 4-7개월 누적 (출구 기준 우선)

Phase 3: 고위험 도메인 - 백엔드 및 배포

목표: Firebase 백엔드 및 배포 자동화

활성화 에이전트:

- Firebase Specialist
- Architect (시스템 설계 검토)

출구 기준:

- ☒ 백엔드 개발 태스크 완료율 >80%
 - o Firebase Functions, Firestore 스키마 변경
- ☒ 동적 가드레일 완전 구현
 - o 런타임 이상 행위 탐지, 행동 기반선 설정

- ABAC: FunctionDeployer, DatabaseMigration 등 위험 도구 다중 조건 검증
- ☒ 에이전트 생성 PR 변경 실패율 ≤ 인간 개발자 기준선
 - DORA elite: <15%
- ☒ 3단계 게이트 HITL 시스템 운영
 - 계획 승인, 테스트 통과 확인, 최종 PR 검토
- ☒ 구조화된 피드백으로 최소 3개 에이전트 성능 개선 입증
 - 예: Flutter Specialist ModernColors 위반 15% → 2%

세르파 특화 검증:

- 포인트 인플레이션 감지 및 방지 시스템 작동
- 게임 밸런스 안정성 유지 (플레이어 피드백 이슈 감소)

예상 기간: 7-12개월 누적 (출구 기준 우선)

Phase 4: 전략적 자율성 및 지속 개선

목표: 스웸 시스템의 자율적 진화

특징:

- 모든 12개 에이전트 완전 활성화
- Skill 승격 수명주기 자동화
- 자가 치유 워크플로우 그래프 운영
- 에이전틱 RAG 고급 패턴 적용

출구 기준:

- ☒ 전체 개발 속도 복리 성장 달성
 - 월별 생산성 증가율 >10%
- ☒ 4중 동기부여 엔진 균형도 유지
 - 각 엔진별 투자 시간 편차 ±20% 이내
- ☒ 비용 효율성 지속 개선
 - 기능 포인트당 LLM 비용 감소 추세
- ☒ 품질 지표 안정화
 - 전체 품질 점수 >4.5/5
 - PR 거절률 <10%
- ☒ 세르파 앱 품질 목표 달성
 - ModernColors 100% 준수

- 게임 밸런스 안정
- 사용자 피드백 개선

지속 상태: 프로덕션 운영 및 지속적 개선

Part 4: 기존 설계와의 비교

4.1 비교표

항목	버전 1.0	버전 2.0	개선 효과
에이전트 수	11개 (4 Tiers)	12개 (5 Tiers, Tier 0 신설)	메타 레벨 감독 추가
오케스트레이션	순수 계층형 (관리형 계층 구조)	하이브리드 (계층+그래프, P2P 협업)	병목 제거, 40% 속도 향상
메모리 관리	공유 컨텍스트 (개념적)	3계층 협업 메모리 (구현 수준)	데이터 무결성, 충돌 방지
보안	정적 규칙 (샌드박싱, 최소 권한)	동적 가드레일 (런타임 탐지, ABAC)	내부 오작동 84% 감소
관측성	기본 로깅 언급	3계층 대시보드 (비즈니스 가치 측정)	의사결정 질 측정 가능
피드백	비정형 코멘트	4단계 구조화된 파이프라인	정량적 개선, 학습 가능
로드맵	시간 기반 (0-6개월, 6-18개월)	메트릭 기반 (Phase 0-4, 출구 기준)	성과 기반 진급
통신 프로토콜	JSON 사용 제안	표준화된 스키마 (TaskAssign 등)	프로토콜 드리프트 방지
RAG	벡터 DB 기반 RAG	에이전틱 RAG (계획-검색-교정)	검색 품질 향상
워크플로우	선형적 단계	상태 기반 그래프 (자가 치유 루프)	순환적 개발 지원
HITL	최종 PR 검토	3단계 게이트 (계획, 테스트, 최종)	조기 오류 발견

4.2 주요 개선 사항 상세

1. System Auditor 추가 (Tier 0 신설)

- Before: 스웸 시스템 자체의 건강 모니터링 부재
- After: 비용, 성능, 품질 지속 모니터링 및 보고
- 효과: "보이지 않는 실패" 방지, 조기 경고

2. 하이브리드 오케스트레이션

- Before: 모든 통신이 Orchestrator를 거침 → 병목
- After: 계획 단계는 계층형, 실행 단계는 P2P
- 효과: 통신 지연 40% 감소, 병렬 처리 향상

3. 3계층 협업 메모리

- Before: "공유 컨텍스트" 개념만 제시
- After: 1계층(개인), 2계층(공유 단기), 3계층(공유 장기) + RBAC
- 효과: 데이터 충돌 0건, 메모리 포이즈닝 방지

4. 동적 가드레일

- Before: 정적 규칙 (샌드박스, 최소 권한)
- After: 행동 기반선 + 런타임 탐지 + ABAC
- 효과: 내부 오작동 84% 감소 (Anthropic 사례)

5. 3계층 관측 대시보드

- Before: 기본 로깅 언급
- After: 시스템 상태 + 에이전트 성능 + 비즈니스 가치
- 효과: 에이전트 활동 → 비즈니스 가치 직접 측정

6. 구조화된 피드백 파이프라인

- Before: PR 코멘트 (비정형)
- After: 4단계 (UI → DB → 분석 → 통찰력)
- 효과: 정량적 개선, 에이전트 학습 가능

7. 메트릭 기반 Phase Gates

- Before: 시간 기반 (0-6개월, 6-18개월)
- After: 성과 기반 출구 기준 (Phase 0-4)
- 효과: 기반 미흡 시 진급 방지, 안정성 확보

Appendix

A. 참고 문서

원본 문서:

1. [C:\sherpa_app\project\sherpa_multi_agent_architecture.md](#) (버전 1.0)
 - 11개 에이전트 구조
 - 세르파 특화 책임 정의

2. C:\sherpa_app\project\세르파 앱 에이전트 시스템 설계 검토.pdf

- 2025년 최첨단 에이전트 아키텍처 패턴
- 5가지 고도화 전략
- 프로덕션급 운영 가이드

Claude 문서:

- C:\sherpa_app\CLAUDE.md: 세르파 앱 개발 가이드
- C:\Users\student\.claude\PERSONAS.md: SuperClaude 페르소나 시스템

B. 주요 개념 용어집

용어	설명
하이브리드 오케스트레이션	계획 단계는 계층형, 실행 단계는 그래프/P2P 협업
협업 메모리 아키텍처	3계층 메모리 (개인/공유 단기/공유 장기) + RBAC
동적 가드레일	런타임 이상 행위 탐지 + 행동 기반선 + ABAC
에이전틱 RAG	계획 → 검색 → 반성 → 교정 루프 (자가 교정)
Skill 승격 수명주기	성공 패턴 → 코드화 → Skill 등록 → 프롬프트 최적화
Phase Gates	메트릭 기반 출구 기준으로 단계 진급 결정
RBAC	역할 기반 접근 제어 (메모리, 도구)
ABAC	속성 기반 접근 제어 (다중 조건 검증)
DORA 지표	변경 리드 타임, 변경 실패율, 평균 복구 시간
복리적 개발 속도	월별 생산성 증가율 >10% (1년 후 3배 향상)

C. 세르파 핵심 규칙 요약

규칙	설명	담당 에이전트
ModernColors 강제	AppColors/RecordColors 금지, ModernColors만 사용	Design System Guardian
Provider 순서	Level 0 → Level 1 → Level 2 → Level 3 엄수	State Management Specialist
questProviderV2	questProvider 레거시 금지, V2만 사용	State Management Specialist
Navigation ID 기반	객체 전달 금지, ID만 전달	Flutter Specialist
게임 밸런스 검증	공식 변경 시 반드시 시뮬레이션 실행	Game Logic Analyst
포인트 경제 규칙	1 point = 1 won, 10% 수수료, 10,000 최소 출금	Point Economy Analyst
4중 엔진 균형	각 엔진별 투자 시간 편차 ±20% 이내	System Auditor

🔗 결론

세르파 개발 스위트 2.0은 버전 1.0의 견고한 기반에 2025년 최첨단 에이전트 아키텍처 패턴을 통합하여, 프로덕션 환경에서 안정적으로 운영 가능한 멀티 에이전트 시스템으로 발전했습니다.

핵심 성과:

- ☒ 12개 에이전트 (System Auditor 추가)
- ☒ 5가지 고도화 전략 완전 적용
- ☒ 세르파 특화 요소 깊이 반영
- ☒ 메트릭 기반 Phase Gates 로드맵

기대 효과:

- 복리적 개발 속도 (월 +10%, 1년 후 3배)
- 변경 실패율 <15% (DORA elite)
- 게임 밸런스 + UX 일관성 + 개발 속도 동시 달성
- 기술적 해자(Moat) 구축

다음 단계: Phase 0 (기반 준비) 시작

- 3계층 협업 메모리 구현
- 통신 프로토콜 정의
- 관측 대시보드 구축
- Orchestrator + System Auditor 구현

세르파 개발 스위트 2.0을 통해 AI 에이전트가 단순한 '조수'를 넘어 진정한 '협업자'로 진화하는 패러다임을 실현하겠습니다.

문서 종료